

Real-Time Credit Card Fraud Detection Pipeline

The problem

Fraud cases in credit card transaction data are rare — about 0.1% to 0.2% of all transactions. That sounds small, but it breaks standard models. A model can label every single transaction as "legitimate" and still hit over 99% accuracy, while catching zero actual fraud. That's a false sense of success.

My goal was to build a system that gets past this trap and actually detects fraud, not just optimizes a misleading number.

What I built

Data & preprocessing. I used the standard credit card fraud dataset. To deal with the class imbalance, I applied SMOTE to generate synthetic samples for the fraud class, so the model would actually learn fraud patterns instead of defaulting to "legit" every time.

Model architecture. I moved past simple classifiers to a Deep ANN. I added Dropout layers to prevent overfitting and Batch Normalization to stabilize training. This handled the complexity of the feature space better than a decision tree could.

Pipeline & deployment. Built end-to-end:

- FastAPI for inference requests.
- Redis as a caching layer, bringing latency down to ~50ms — important for a real-time financial system.
- A Streamlit dashboard for monitoring live traffic and triggering alerts on high-risk transactions.

Evaluation. I dropped accuracy as a metric — it's misleading here. I optimized for Precision-Recall AUC and Recall instead, since missing actual fraud costs far more than a false positive.

What didn't work

- **Aggressive undersampling.** My first approach balanced the classes to 50/50 by undersampling. It worked on paper, but the model lost so much information from the legitimate-transaction majority that it started flagging real transactions as fraud constantly. The false positive rate made it unusable.
- **Default threshold (0.5).** When I moved to the Deep ANN, I left the risk threshold at the default 0.5. It caught almost nothing — the model was too conservative. I had to lower the threshold significantly to catch more fraud, accepting a slightly higher false positive rate as the tradeoff.
- **Caching everything in Redis.** I originally planned to cache every single inference result. That was unnecessary overhead and wasted memory for the traffic levels I was

simulating. I reworked it to only cache high-frequency features and model metadata, which kept latency low without the waste.

What it actually does

After tuning the threshold, the final numbers:

- **ROC-AUC: 0.958** — strong separation between fraud and legitimate transactions.
- **Threshold set at 0.90:** F1 score of 0.77, Precision 0.738, Recall 0.806. Lower thresholds caught more fraud but spiked false alerts — 0.90 was the most practical operating point.
- **Latency: ~50ms end-to-end**, consistent, thanks to Redis caching and offloading transaction metadata to memory.
- **Built for high-concurrency traffic** — the pipeline stayed performant under simulated peak load.

What I'd change next

1. **Sequence modeling.** Right now the model looks at individual transactions. Adding user behavior data — location changes, spending pattern shifts over the last 24 hours — and moving to an RNN/LSTM or Transformer architecture would let the system evaluate risk across a sequence of transactions, not just one at a time.
2. **Handling concept drift.** Financial behavior changes over time, and model performance degrades as a result. I'd build an incremental learning or automated retraining pipeline so the model keeps updating itself with new data instead of going stale.
3. **Explainability.** Beyond the Streamlit dashboard, I'd add SHAP or LIME so the system can explain *why* it flagged a transaction as fraud, not just that it did. That's the difference between "there's a risk" and something a fraud analyst can actually act on with confidence.

About Me

I'm a backend AI engineer. I build AI pipelines that turn messy, unstructured data into something reliable enough to use in production — whether that's genomic research or financial fraud detection. My focus is closing the gap between a model that works in a notebook and a system that works in the real world.

Contact

- GitHub: github.com/caglaeren
- LinkedIn: linkedin.com/in/cagla-eren